

Q.No:2

Problem Description

You are given an N-dimensional array A. You can execute an operation that removes the greatest and smallest elements from the array and replaces them with their difference. As a result, the array's size will drop by one after each operation. You are assigned Q jobs, each of which contains an integer K. After K operations, you must provide the total of all the items in the array for each task.

Constraints:

$$2 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

$$0 \leq A[i] \leq 10^9$$

$$0 \leq K < N$$

Input:

First line contains two space-separated integers N and Q, denoting the number of elements in array and number of queries respectively.

Next line contains N space-separated integers denoting elements of the array.

Next Q lines contain a single integer K.

Output:

For each task, print answer in a new line.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
3 2 1 5 4
1
2
```

EXPECTED OUTPUT

```
13
9
```

Test Case 2

INPUT (STDIN)

```
5 3
13 12 11 15 14
1
2
3
```

EXPECTED OUTPUT

```
43
35
15
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<n;i++)
```

Code:

```
#include <stdio.h>

#define MAX 200005

long long minHeap[MAX];
long long maxHeap[MAX];

int minSize = 0;
int maxSize = 0;
void minPush(long long x)
{
    int i = minSize++;
    minHeap[i] = x;
```

```
while (i > 0)
{
    int p = (i - 1) / 2;

    if (minHeap[p] <= minHeap[i])
        break;

    long long temp = minHeap[p];
    minHeap[p] = minHeap[i];
    minHeap[i] = temp;

    i = p;
}

long long minPop()
{
    long long result = minHeap[0];

    minSize--;

    if (minSize > 0)
    {
        minHeap[0] = minHeap[minSize];

        int i = 0;

        while (1)
        {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int small = i;

            if (left < minSize &&
                minHeap[left] < minHeap[small])
                small = left;

            if (right < minSize &&
                minHeap[right] < minHeap[small])
                small = right;

            if (small == i)
                break;

            long long temp = minHeap[i];
            minHeap[i] = minHeap[small];
            minHeap[small] = temp;

            i = small;
        }
    }
}
```

```
return result;
}

void maxPush(long long x)
{
    int i = maxSize++;

    maxHeap[i] = x;

    while (i > 0)
    {
        int p = (i - 1) / 2;

        if (maxHeap[p] >= maxHeap[i])
            break;

        long long temp = maxHeap[p];
        maxHeap[p] = maxHeap[i];
        maxHeap[i] = temp;

        i = p;
    }
}

long long maxPop()
{
    long long result = maxHeap[0];

    maxSize--;

    if (maxSize > 0)
    {
        maxHeap[0] = maxHeap[maxSize];

        int i = 0;

        while (1)
        {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int big = i;

            if (left < maxSize &&
                maxHeap[left] > maxHeap[big])
                big = left;

            if (right < maxSize &&
                maxHeap[right] > maxHeap[big])
                big = right;

            if (big == i)
                break;

            long long temp = maxHeap[i];
            maxHeap[i] = maxHeap[big];
            maxHeap[big] = temp;

            i = big;
        }
    }

    return result;
}
```

```
break;

long long temp = maxHeap[i];
maxHeap[i] = maxHeap[big];
maxHeap[big] = temp;

i = big;
}
}

return result;
}

int main()
{
    int N, Q;

    scanf("%d %d", &N, &Q);

    minSize = 0;
    maxSize = 0;

    long long sum = 0;

    long long A[MAX];

    for (int i = 0; i < N; i++)
    {
        scanf("%lld", &A[i]);

        sum += A[i];

        minPush(A[i]);
        maxPush(A[i]);
    }

    long long answer[MAX];

    for (int q = 0; q < Q; q++)
    {
        int K;

        scanf("%d", &K);
        minSize = 0;
        maxSize = 0;
        sum = 0;

        for (int i = 0; i < N; i++)
        {
            minPush(A[i]);
            maxPush(A[i]);
            sum += A[i];
```

```
}  
  
for (int i = 0; i < K; i++)  
{  
    long long smallest = minPop();  
    long long largest = maxPop();  
  
    long long difference = largest - smallest;  
  
    sum = sum - smallest - largest + difference;  
  
    minPush(difference);  
    maxPush(difference);  
}  
  
answer[q] = sum;  
}  
  
printf("Ouptut:\n");  
for (int i = 0; i < Q; i++)  
{  
    printf("%lld\n", answer[i]);  
}  
  
return 0;  
}
```

Output:

Test_case :2

```
5 2  
3 2 1 5 4  
1  
2  
Ouptut:  
13  
9  
aadityajaiswal@Aadityas-
```

Test_case :2

```
aadityajaiswal@Aadityas-M  
5 3  
13 12 11 15 14  
1  
2  
3  
Ouptut:  
43  
35  
15  
aadityajaiswal@Aadityas-M
```

Q.No: 3

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m : the number of cities and flights. The cities are numbered $1, 2, \dots, n$.

After this, there are m lines describing the flights. Each line has two integers a and b : there is a flight from city a to city b . All flights are one-way flights.

Output

First print an integer k : the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
2 3
3 1
1 4
3 4
```

EXPECTED OUTPUT

```
1
4 1
```

Test Case 2

INPUT (STDIN)

```
4 5
1 3
2 1
1 2
2 4
1 4
```

EXPECTED OUTPUT

```
2
4 3
3 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m--)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100005

typedef struct Edge {
    int to;
    int next;
} Edge;
```